

Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey



Course Name: Computer Architecture Lab

Course Number and Section: **14:332:331:01**

Experiment: [Experiment # 4 – *RISC-V Registers & Procedures*]

Lab Instructor: Xin Li

Date Performed: 10/25/2024

Date Submitted: 11/8/2024

Submitted by: Samuel Habib 217007500



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

Computer Architecture and Assembly Lab (14:332:333)
Fall 2024

Lab 4 *RISC-V Registers & Procedures*

Goals

In Lab 4, we will discuss the following topics:

- Review the Venus simulator, including the environmental call and assembler directives.
- RISC-V calling conventions, e.g., register usage.
- Introduction to the function/procedure in RISC-V.

Instructions

- All the programs must be compiled by Venus RISC-V Simulator. Please note that the simulator only supports 32-bit instruction, and we do not consider overflow.
- Once you complete the assignment, please upload your source code and PDF lab report on Canvas.
- **Comment your code clearly. You can use the mark "#" to add comments.**
- Please copy and paste your code into the lab report.

Introduction

Venus Simulator Review

Environmental call (ecall)

Just as 'printf' is used in the C language to display content on the console, 'ecall' serves a similar purpose. Here, the register `'a0'` (also known as `'x10'`) holds the function code, while `'a1'` (or `'x11'`) contains the actual content we aim to display.

Assembler directives

Besides instructions in assembler format, an assembler also accepts so-called assembler directives.

- `.text` – Where the code starts. Each assembler instruction can start with a label such as `main:`.
- `.data` – data segment, where you can initialize data. Also, data can be addressed by using a label.



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

RISC-V Calling Conventions in Register Usages

In RISC-V programming, the calling convention is a standard that defines how functions/procedures receive parameters and return values. It also determines which registers are used for what purpose and who (caller or callee) is responsible for saving and restoring the contents of registers.

ra (x1): return address. No need to save & restore
a0-a7 (x10–x17): registers for procedures
a0, a1 (x10, x11): procedure arguments/return values
a2-a7 (x12–x17): procedure arguments. No need to save & restore
t0-t6 (x5–x7, x28–x31): temporary registers. Not preserved by the callee. No need to save & restore
s0-s11 (x8, x9, x18–x27): saved registers. Must be preserved by the callee. Need to save & restore

Introduction to RISC-V Functions/Procedures

The function/procedure is a stored subroutine that performs a specific task based on the provided parameters.
Procedure call: jump-and-link, e.g., **`jal ra, ProcedureLabel`**

- Puts the address of following instruction to **ra**
- Jumps to target address **ProcedureLabel**

Procedure return: jump-and-link register, e.g., **`jalr x0, 0(ra)`**

- Like **jal**, but jumps to (0 + address in **ra**)
- Use **ra** as the destination register (rd) (**x0** cannot be changed).

Steps required:

1. Place parameters in registers (**a0** to **a7**)
2. Transfer control to procedure
3. Acquire storage resources needed for procedure
4. Perform procedure's operations
5. Place results in registers for the caller (**a0**, **a1**)
6. Return control to the point of origin (address in **ra**)



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

Exercise 1 RISC-V Calling Conventions [40 pts]

Please read the C program below and answer the following questions.

```c
//ex1.c
#include <stdio.h>

int source[] = {2, 4, 6, 8, 10, 12, 0};

int main() {
    int k = 0;
    int rem = 100;

    while (source[k] != 0) {
      rem = rem - source[k];
      k++;
    }

    printf("%d", rem);

    return 0; //no need to translate this line of code
}
```

1.1 [5 pts] In RISC-V, which registers would you use to send data or inputs into procedures when programming?

X10-x17

1.2 [5 pts] In RISC-V, which register(s) should hold the results that are produced by procedures?

X10 and x11



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

1.3 [5 pts] If you are calling a procedure using the `'jal'` instruction and you expect to return from that procedure, is it necessary to save the values stored in the register `'ra'` before using `'jal'`? Please write down your explanation.

It's only necessary to save the return address if the procedure you are running will return other functions, nested procedures. So in this case it's **not** necessary.

1.4 [5 pts] If you're modifying the value of `'s0(x8)'` within a procedure and intend to use this updated value every time the procedure is called. Is it necessary to save the values stored in `s0`? Please write down your explanation.

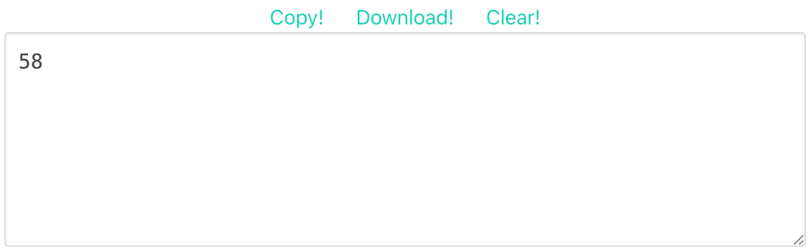
Any changes made to the s0 register within the procedure should not affect its values from the perspective of the caller. The register s0 is supposed to retain its value across function calls, a procedure that modifies s0 will have to store it at the beginning and then restore it before the function is returned. Therefore, it **is necessary.**

1.5 [20 pts] Please convert the C program into a RISC-V program by following calling conventions. The RISC-V program should work the same way as the C program at the function/procedure level, which means the loop from the original program should be maintained. You might need to define a `'main'` procedure and a `'loop'` procedure to properly answer this question.

Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

In your report, please include the following:

(1) A screenshot of your program's output.



(2) Your annotated program code, with comments for clarity and understanding. **Please copy and paste the code into your report.**

```
.data
  source: .word 2, 4, 6, 8, 10, 12, 0    # array values
.text
.globl main
main:
  li x5, 0              # k = 0
  li x6, 100            # rem = 100
  la x7, source         # load array address
loop:
  slli x8, x5, 2        # k * 4 for offset
  add x8, x8, x7        # get source[k] address
  lw x9, 0(x8)          # load source[k] value
  beq x9, x0, end_loop    # exit if source[k] = 0
  sub x6, x6, x9        # rem = rem - source[k]
  addi x5, x5, 1        # k++
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

```
    bne x9, x0, loop        # loop if not zero
end_loop:
    li x10, 1               # print syscall
    mv x11, x6              # move rem to x11
    ecall
    li x10, 10              # exit syscall
    ecall
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

Exercise 2 Writing RISC-V Procedures [60 pts]

2.1 [30 pts] Please write a program in RISC-V that completes the following requirements:
In the **`.data`** directive:

- Define the integer (in 32-bit) variables **`x`** and **`y`**.

In the **`.text`** directive:

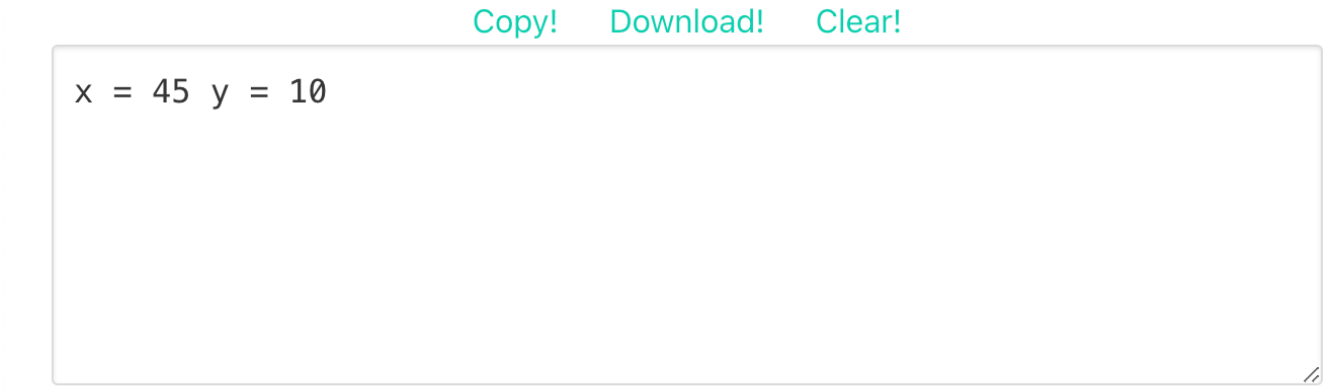
- The **`main`** procedure is designed to retrieve the input values from the **`.data`** directive, and enter the **`branch_1`** and the **`branch_2`** respectively. Also, the main procedure prints the computed **`x`** and **`y`** in the console using ecall after calling the **`branch_1`** and the **`branch_2`**. For example, the output could be formatted as: **`"x = 20 y = 5".`**
- In the **`branch_1`** procedure, if **`x < 20`**, compute **`5*x`**, else do nothing.
- In the **`branch_2`** procedure, if **`y`** is negative, compute **`-y`**, else do nothing. You can get the opposite value by calculating **`0-y`**.



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

In your report, please include the following:

(1) A screenshot of your program's output. The output should be based on your own input



(2) Your annotated program code, with comments for clarity and understanding. **Please copy and paste the code into your report.**

```
.data
   x: .word 9          # initial value of x
   y: .word 10         # initial value of y
   msg1: .string "x = "   # output message for x
   msg2: .string " y = "  # output message for y


.text
.globl main
main:
   la x5, x            # load address of x
   lw x6, 0(x5)        # load value of x
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

```
la x7, y            # load address of y
lw x8, 0(x7)        # load value of y
jal branch_1        # call branch_1
sw x6, 0(x5)        # store updated x
jal branch_2        # call branch_2
sw x8, 0(x7)        # store updated y
li x10, 4           # print string syscall
la x11, msg1
ecall
li x10, 1           # print int syscall
mv x11, x6
ecall
li x10, 4           # print string syscall
la x11, msg2
ecall
li x10, 1           # print int syscall
mv x11, x8
ecall
li x10, 10          # exit syscall
ecall
```

```
branch_1:
   li x9, 20          # load 20 for comparison
   bge x6, x9, skip1  # if x >= 20, skip
   li x9, 5           # load 5 for multiplication
   mul x6, x6, x9     # x = 5 * x
skip1:
   ret                # return to main

branch_2:
   bge x8, x0, skip2  # if y >= 0, skip
   sub x8, x0, x8     # y = 0 - y (make positive)
skip2:
   ret                # return to main
```

2.2 [30 pts] In this question, we define an expression in the **`exp`** procedure:

$$\texttt{exp(x)} = x^4 - 5x + 10$$

Please write a program in RISC-V that completes the following requirements:

- The **`main`** procedure is to call the **`exp`** procedure and print the output in the console. You can assume the input **`x`** will always be positive, and hardcode the value of **`x`** in the main procedure.
- The **`exp`** procedure will compute and return the expression. Procedure **`exp`** must call the following additional procedures:
    - Call the **`power`** procedure to calculate x^n. You can assume that **`n`** will be positive.
    - Call the **`times`** procedure to calculate **`m*x`**.
    - Compute the result of $x^4 - 5x + 10$.



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

In your report, please include the following:

(1) A screenshot of your program's output. The output should be based on your own input

Copy! Download! Clear!

```
246
```

(2) Your annotated program code, with comments for clarity and understanding. **Please copy and paste the code into your report.**

```
.text
.globl main
main:
  li x5, 2          # input x = 4
  mv x10, x5        # copy x to argument register
  jal exp           # call exp procedure

  mv x11, x10       # move result to print register
  li x10, 1         # print int syscall
  ecall
```

```
  li x10, 10          # exit syscall
  ecall

exp:
  addi sp, sp, -16    # adjust stack
  sw ra, 12(sp)       # save return address
  sw x5, 8(sp)        # save x

  li x6, 4            # n = 4 for x^4
  jal power           # calculate x^4
  mv x7, x10          # save x^4 result

  lw x5, 8(sp)        # restore x
  li x6, 5            # m = 5 for 5x
  jal times           # calculate 5x
  mv x8, x10          # save 5x result

  sub x10, x7, x8     # x^4 - 5x
  addi x10, x10, 10   # add 10
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

```
  lw ra, 12(sp)         # restore return address
  addi sp, sp, 16       # restore stack
  ret

power:                  # calculates x^n
  li x10, 1             # result = 1
power_loop:
  beq x6, x0, skip_pow  # if n == 0, done
  mul x10, x10, x5      # result *= x
  addi x6, x6, -1       # n--
  j power_loop
skip_pow:
  ret

times:                  # calculates m*x
  mul x10, x5, x6       # result = x * m
  ret
```